Title: When Does Batch Prompting Help Large Language Models?

Abstract / Overview:

Batch prompting processes multiple inputs in one LLM inference call, reducing token use and inference time as batch size increases (Cheng et al., 2023, https://arxiv.org/abs/2301.08721). Yet its effect on quality is inconsistent: performance varies by task, model, and batch size; long or complex inputs suffer drops; and no reliable method predicts whether batching will help or hurt a task.

Work on minimal-edit Grammatical Error Correction (GEC) found that batching reduces overcorrection and improves precision-weighted F0.5 scores across LLM families (Karpo and Chernodub, 2026, https://arxiv.org/abs/2609.10810). The proposed explanation is attention dilution: as inputs share the context window, the model distributes limited attention among them, potentially regularizing behavior. This mechanism remains unconfirmed, and the conditions under which it helps or harms performance are unknown.

This Ms. Sc. thesis will:
1. Characterize batch prompting across tasks, models, and batch sizes.
2. Test whether attention dilution explains quality changes and propose another explanation through open-weight LLM analysis if needed.
3. Develop an unsupervised criterion predicting whether batching will improve or degrade performance on a task.
4. Translate findings into recommendations and an open-source tool.

The criterion will use unlabeled inputs, model-internal signals such as attention statistics and output-distribution shifts, and task characteristics including input/output length, complexity, and reasoning style. It will require neither labeled examples nor supervised training on the target task. The goal is to help practitioners decide whether and how to use batching before a costly evaluation.

References

1. Cheng, Z., Kasai, J., and Yu, T. (2023). "Batch Prompting: Efficient Inference with Large Language Model APIs." Proceedings of EMNLP 2023: Industry Track, pp. 792–810. https://arxiv.org/abs/2301.08721
2. Karpo, K. and Chernodub, A. (2026). "Larger Context Window, Fewer Overcorrections: Optimizing Prompts and Batching for Minimal-Edit Grammatical Error Correction." Findings of EMNLP 2026. https://arxiv.org/abs/2609.10810

Main Objectives / Research Questions:
The primary objective is to determine when and why batch prompting changes LLM output quality, and to develop a practical, unsupervised method for recommending whether to use batch prompting, which batch size to use, and when batching is likely to harm performance.

Key Research Questions (RQs)
To achieve this objective, the research will address the following questions:

RQ1: Characterizing the Batching Effect
How does batch prompting affect quality across different tasks, models, and batch sizes? The study will examine which task- and model-level properties are associated with performance gains or losses.

RQ2: Testing the Attention-Dilution Hypothesis
Does attention dilution cause the quality changes associated with batch prompting? Controlled experiments on open-weight models will measure how batch size affects attention entropy, head-level attention concentration, cross-input attention leakage, and the relationship between these signals and downstream quality. The analysis will test whether attention changes mediate the effect of batching rather than merely correlate with it.

RQ3: Predicting the Effect Without Target-Task Labels
Can unlabeled inputs and model-internal signals predict the direction and magnitude of the batching effect? The study will evaluate which unsupervised signals are most informative, whether the criterion can distinguish beneficial from harmful batching, how well it transfers to unseen tasks and model families, and whether it can recommend an appropriate batch size.

RQ4: Practical Recommendations and Tooling
Can the proposed criterion be implemented as a practical tool? Given unlabeled task inputs and a target model, the tool should recommend whether to use batch prompting, a suitable batch size, and the expected direction and confidence of the quality change. Its recommendations will be compared with full supervised evaluations on held-out tasks.

Expected Outcomes
The project is expected to deliver:

- A systematic evaluation of batch prompting across tasks, models, and batch sizes.
- An unsupervised criterion for predicting whether batching will help or hurt a new task.
- Evaluation of the criterion on held-out tasks and model families.
- An open-source tool for generating batching recommendations.

Prerequisites
Mandatory prerequisites

- Strong Python programming skills.
- Experience designing and running structured experiments.
- A solid understanding of NLP and Transformer architectures.
- Knowledge of self-attention and model-internal representations.
- Practical experience with prompt engineering

Preferred prerequisites

- Mechanistic interpretability or attention probing.
- NLP evaluation metrics and statistical analysis.
- Open-weight model analysis using Hugging Face Transformers.
- Batch prompting or in-context learning.
- Research or software-development projects.
- Git and software-engineering best practices.
- Academic writing.

Data Availability
We are going to use publicly available benchmarks. We will select 4-5 tasks from this list below, each with relevant to it metric:

- Grammatical Error Correction (BEA-2019, CoNLL-2014, evaluated by ERRANT F0.5 and M2).
- commonsense and strategic QA (CommonsenseQA, StrategyQA, evaluated by accuracy);
- arithmetic reasoning (GSM8K, SVAMP, AQuA, AddSub, MultiArith, evaluated by exact match or accuracy);
- NLI and classification (RTE, MNLI, SST-5, evaluated by accuracy);
- table question answering (WikiTQ, evaluated by exact match or denotation accuracy);
- Also, summarization or simplification may also be included, evaluated on standard public benchmarks with task-specific metrics such as ROUGE or SARI.

3.

Access to infrastructure for running open-weight LLM models such as Qwen-3-8B or Gemma-4-12B is highly desirable (we can not provide it).